PROYECTO INTEGRADOR 2 - SISTEMA DE GESTIÓN DE ASIENTOS DE CINE



FACULTAD DE CIENCIAS Y TECNOLOGÍAS DE LA INFORMACIÓN

MATERIA: ESTRUCTURA DE DATOS

INTEGRANTES:
VALERIA BERENICE ALVARADO CASTILLO
ROSALIA CHAVEZ LAGUNAS

MAESTRO: ROBERTO ROSARIO

FECHA: 8 DE NOVIEMBRE 2026

1. INTRODUCCIÓN

El presente proyecto integrador aborda el desarrollo e implementación de un sistema de consola en lenguaje C# diseñado para digitalizar la administración y control de asientos dentro de una sala de cine. El objetivo primordial es erradicar errores comunes en las reservaciones manuales mediante el uso de estructuras de datos bidimensionales (matrices), permitiendo visualizar en tiempo real el estado de ocupación y disponibilidad de la sala.

2. PLANTEAMIENTO DEL PROBLEMA

Un cine requiere modernizar su proceso de venta y control de boletos. Actualmente, el registro manual de asientos genera duplicidad en las reservaciones y falta de certeza sobre la disponibilidad real de la sala. Se necesita un sistema automatizado que permita visualizar una matriz interactiva de 30 asientos (5 filas por 6 columnas), ofreciendo funciones para reservar, liberar y consultar disponibilidad de forma precisa y eficiente.

3. OBJETIVOS

3.1 Objetivo General: Desarrollar un sistema de consola modular en C# que organice y gestione el estado de 30 asientos en una matriz bidimensional, optimizando la asignación, liberación y consulta de espacios en tiempo real.

3.1 Objetivos Específicos:

1. Representar visualmente la sala de cine mediante una matriz de 5x6 utilizando notaciones claras de disponibilidad (**[O]** para Disponible y **[X]** para Ocupado).
2. Diseñar una arquitectura modular mantenible basada en métodos y estructuras de control claras (como **switch-case** y ciclos **for** anidados).
3. Garantizar un avance progresivo mediante control de versiones en GitHub con la participación equitativa de los dos integrantes del equipo.

4. DÍA 1: ANÁLISIS DEL PROBLEMA

4.1 Identificación de Componentes del Sistema

Entradas (Input):

- Selección de opciones dentro del menú principal (números del 1 al 5).
- Coordenadas numéricas de posición de asiento (Fila de 1 a 5, Columna de 1 a 6).

Procesos (Process):

- Inicialización automatizada de la matriz bidimensional **SalaDeCine[5, 6]** con valores iniciales de disponibilidad (**"[O]"**).

- Evaluación de la opción ingresada por el usuario mediante una estructura de control selectiva **switch-case.**
- Recorrido mediante ciclos anidados **for** para reflejar el estado actualizado de la matriz en la pantalla de la consola.

Salidas (Output):

- Menú interactivo de gestión en consola.
- Representación gráfica en consola de la pantalla del cine y la cuadrícula de asientos ordenada por filas y columnas.
- Mensajes claros de confirmación o alerta de error ante la navegación y reservación.

4.2 Justificación de la Estructura de Datos

Se eligió una matriz bidimensional (**string[,]**) debido a que la distribución física de una sala de cine responde directamente a un plano rectangular compuesto por filas y columnas. Esta estructura de datos permite un acceso directo indexado $O(1)$ a cualquier asiento utilizando simplemente sus coordenadas **[fila, columna]**.

4.3 Alcance y Limitaciones del Proyecto

- **Alcance:** El sistema permite administrar en tiempo real una sala de cine estándar de 30 localidades (5 filas de la A a la E y 6 columnas del 1 al 6), ofreciendo visualización gráfica en consola, reservación, liberación, consulta individual y cálculo estadístico de ocupación (conteo y porcentaje).

- **Limitaciones:** El almacenamiento de datos se realiza en memoria volátil (RAM) durante el tiempo de ejecución, por lo que al cerrar la aplicación la sala se reinicia. El sistema está diseñado para gestionar una única sala y función por ejecución sin pasarela de pagos.

4.4 Especificación de Casos de Uso

- **CU-01 Visualizar Sala:** El usuario selecciona la opción 1; el sistema recorre la matriz e imprime la cuadrícula actualizada con filas A-E y columnas C1-C6.

- **CU-02 Reservar Asiento:** El usuario ingresa fila y columna; el sistema valida existencia y disponibilidad ([O]) para cambiar el estado a ocupado ([X]).

- **CU-03 Liberar Asiento:** El usuario ingresa fila y columna de un asiento ocupado ([X]); el sistema lo restablece a disponible ([O]).

- **CU-04 Consultar Disponibilidad:** El usuario ingresa una coordenada y el sistema informa si está libre u ocupada sin alterar la matriz.

- **CU-05 Consultar Estadísticas:** El sistema recorre la matriz, contabiliza asientos [O] y [X], y calcula el porcentaje de ocupación sobre el total de 30 asientos.

5. DÍA 2: DISEÑO DE LA SOLUCIÓN

5.1 Algoritmo General (Pseudocódigo)

5.1 Algoritmo General (Pseudocódigo)

```
Algoritmo GestionSistemaCine
   Definir SalaDeCine como Matriz[5, 6] de Texto
   Definir opción como Texto
   Definir ejecutando como Lógico

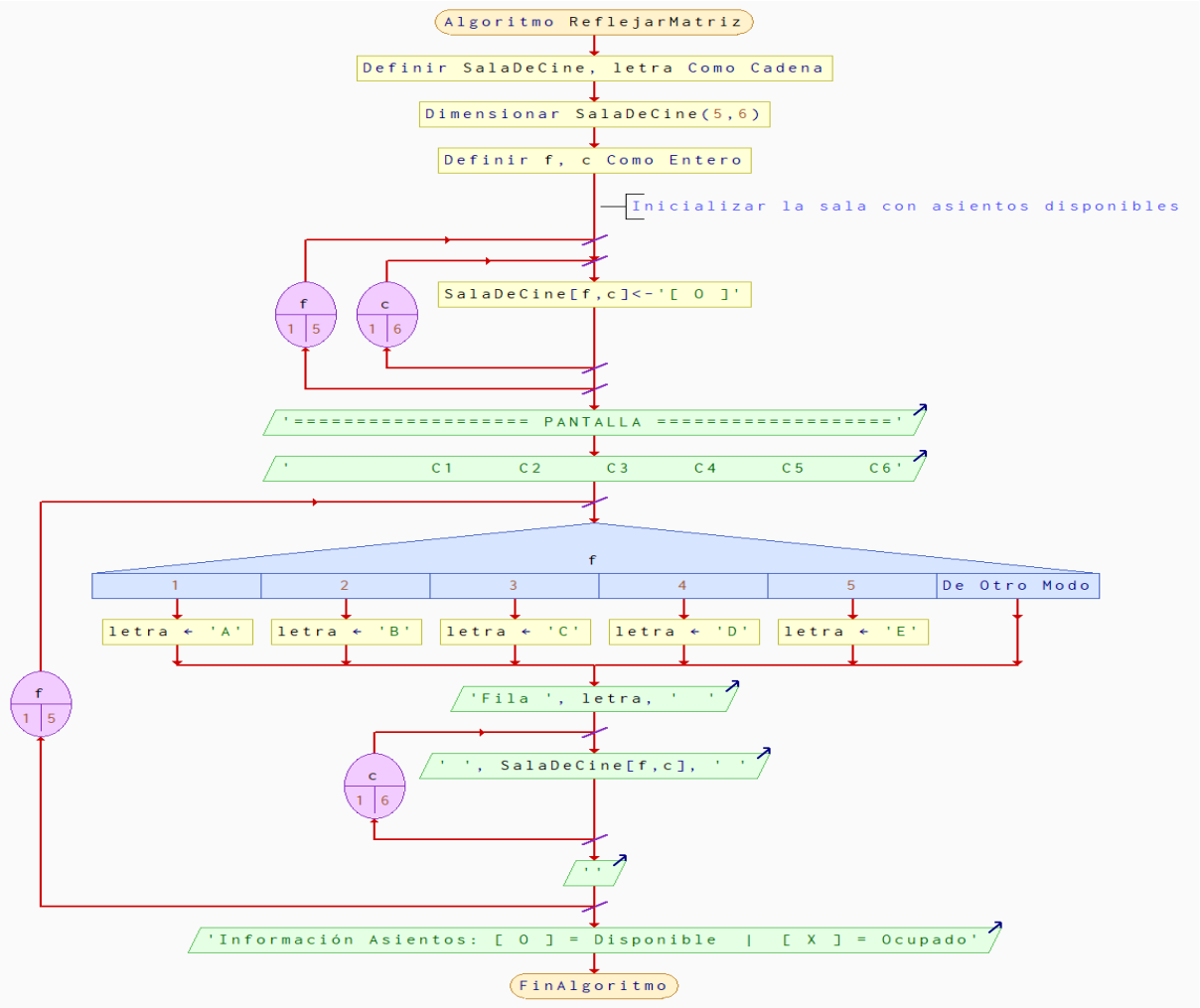
   InicializarSala()
   ejecutando <- Verdadero

   Mientras ejecutando = Verdadero Hacer
      LimpiarPantalla()
      MostrarMenu()
      Leer opcion

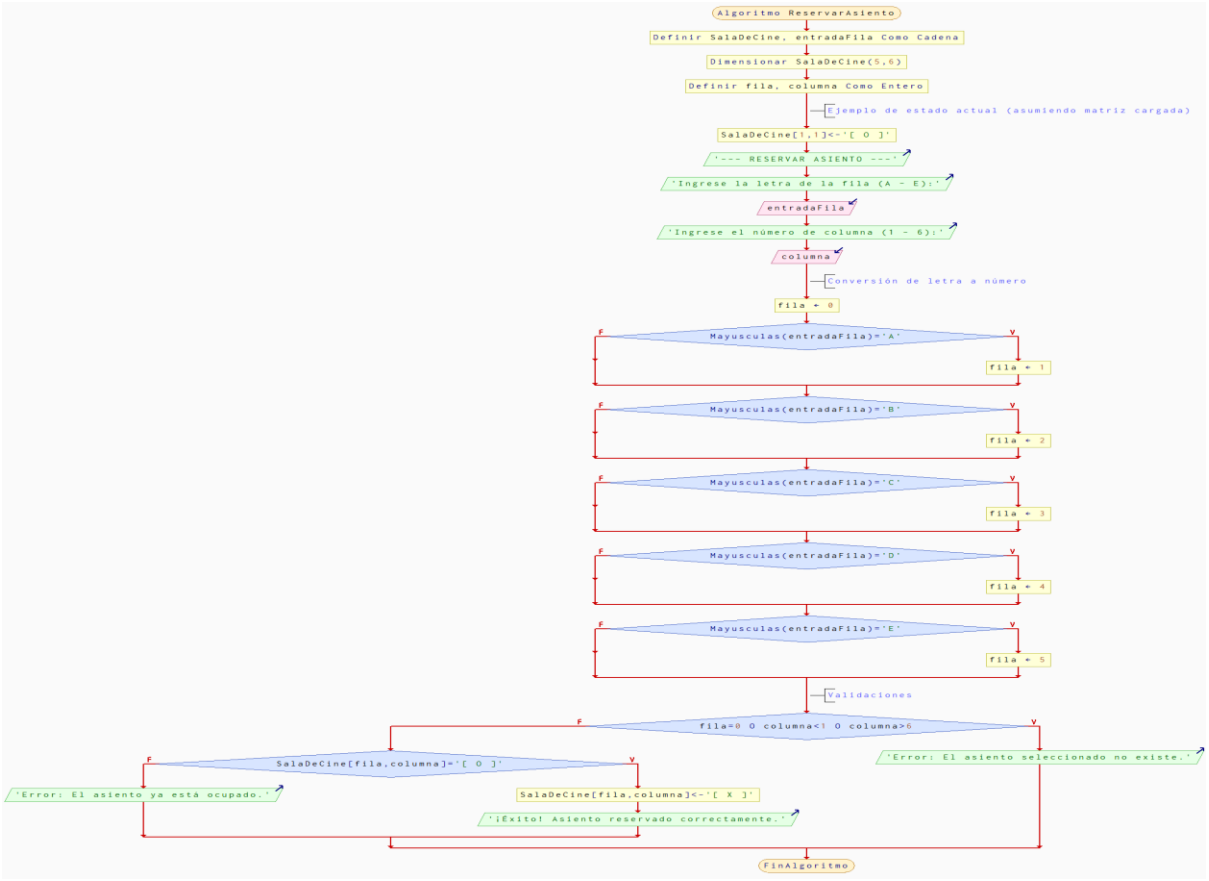
      Según opcion Hacer
         Caso "1":
            ReflejarMatriz()
            Pausar()
         Caso "2":
            ReservarAsiento()
            Pausar()
         Caso "3":
            LiberarAsiento()
            Pausar()
         Caso "4":
            ConsultarDisponibilidad()
            Pausar()
         Caso "5":
            ejecutando <- Falso
            Escribir "¡Saliendo del sistema!"
         De Otro Modo:
            Escribir "Opción no válida. Intente de nuevo."
            Pausar()
      FinSegún
   FinMientras
FinAlgoritmo
```

5.2 Diagramas de Flujo

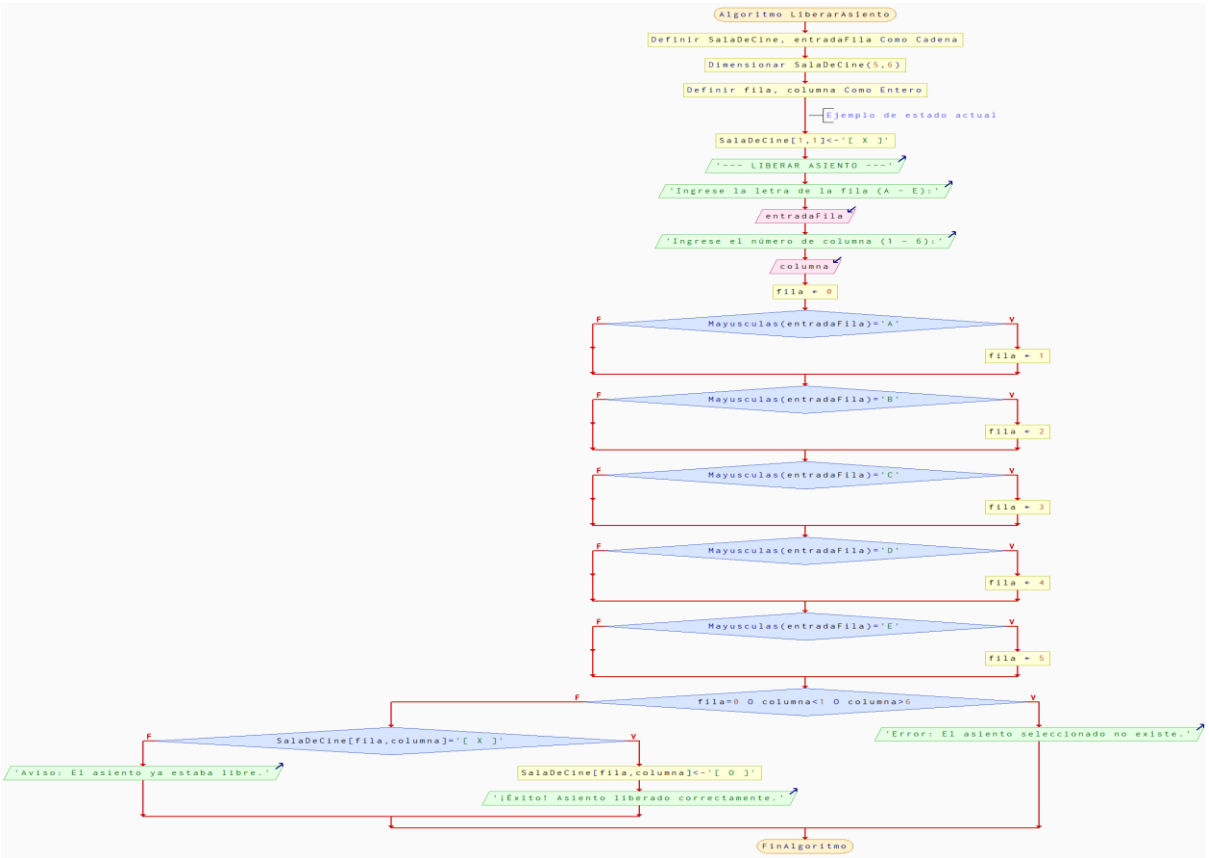
1. Inicializar y Mostrar Estado de la Sala (Notación Pantalla y Asientos)



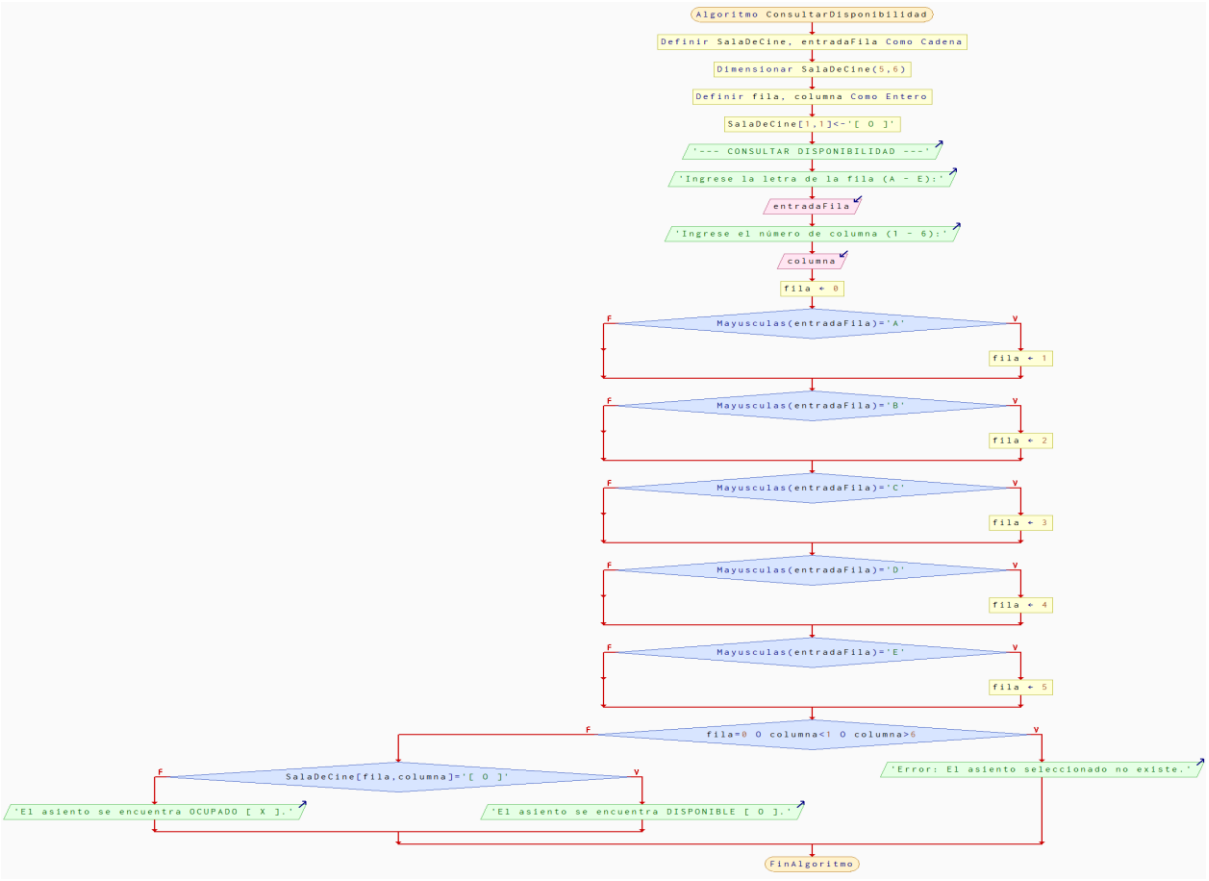
2. Reservar Asiento



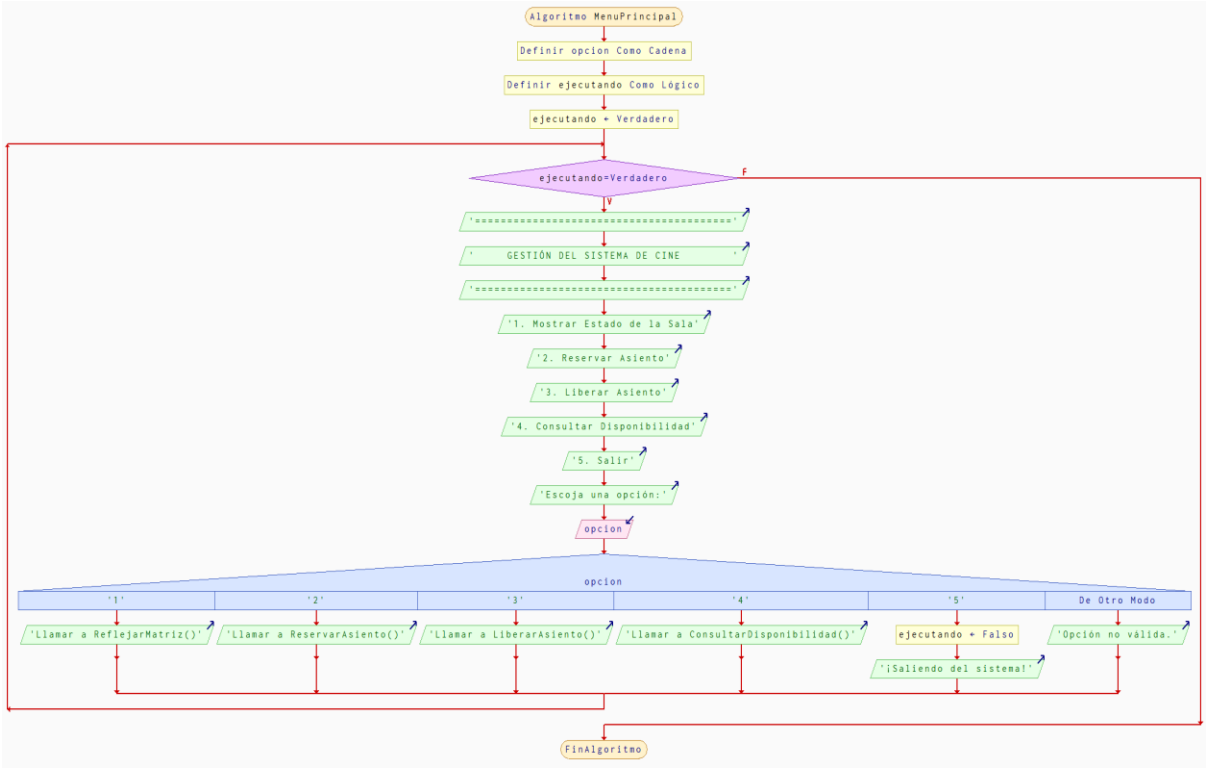
3. Liberar Asiento



4. Consultar Disponibilidad de Asiento



5. Estructura de Control del Menú Principal



6. DÍA 3: DISEÑO DE LA SOLUCIÓN

6.1 Definición de la Estructura de Datos

Para representar la sala de cine se utilizó una **matriz bidimensional estática** de tipo string con dimensiones de 5 filas por 6 columnas, proporcionando una capacidad total de 30 asientos.

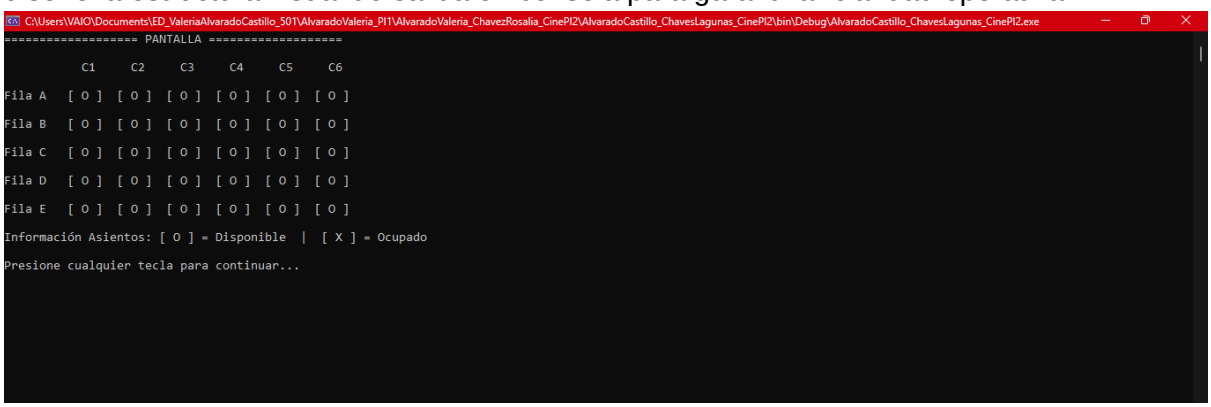
- **Matriz SalaDeCine[5, 6]:** Almacena el estado visual de cada asiento mediante los identificadores "[O]" para asientos disponibles y "[X]" para asientos ocupados.
- **Arreglo de Mapeo LetrasFilas[5]:** Arreglo unidimensional de caracteres (['A', 'B', 'C', 'D', 'E']) que permite mapear los índices numéricos de las filas (0 a 4) a sus etiquetas alfabéticas correspondientes para la vista del usuario.

6.2 Arquitectura Modular de Funciones

El sistema se dividió en métodos auxiliares especializados para garantizar el principio de responsabilidad única y facilitar el mantenimiento del código:

1. **InicializarSala()**: Recorre la matriz mediante dos ciclos for anidados e inicializa todas las posiciones [f, c] con el valor predeterminado "[O]".
2. **ReflejarMatriz()**: Imprime el encabezado de la pantalla, las etiquetas de las columnas (C1 a C6) y recorre la matriz imprimiendo cada fila con su respectiva letra (A a E) y el estado de sus asientos.
3. **ReservarAsiento()**: Solicita la coordenada del asiento (letra de fila y número de columna), valida la existencia del asiento y cambia su estado a "[X]" si se encontraba disponible.
4. **LiberarAsiento()**: Recibe la coordenada del asiento, verifica si está ocupado ("[X]") y restablece su valor a "[O]".
5. **ConsultarDisponibilidad()**: Permite consultar el estado actual de un asiento específico sin modificar su valor en la matriz.
6. **ConvertirLetraAIndice(string letra)**: Método auxiliar que convierte la entrada alfabética del usuario (A-E) al índice numérico entero de la matriz (0-4), retornando -1 en caso de recibir una letra inválida.
7. **Pausar()**: Pausa la ejecución del programa hasta que el usuario presione una tecla, evitando que la consola se limpie automáticamente antes de leer la respuesta.

6.3 Diseño Preliminar de la Interfaz (Mockup de Consola) Antes de la codificación se diseñó la estructura visual de salida en consola para garantizar claridad operativa:



7. DIA 4: IMPLEMENTACIÓN Y FRAGMENTOS DE CÓDIGO EXPLICADOS

7.1 Inicialización y Recorrido de la Matriz

(InicializarSala y ReflejarMatriz): Se emplean dos ciclos for anidados (f de 0 a 4 y c de 0 a 5) para recorrer cada celda de SalaDeCine[5, 6]. Para mejorar la experiencia de usuario, se utiliza el arreglo LetrasFilas que traduce el índice numérico de fila a letras (A–E).

```
// Inicializar la matriz con todos los asientos libres
1 referencia
static void InicializarSala()
{
    for (int f = 0; f < 5; f++)
    {
        for (int c = 0; c < 6; c++)
        {
            SalaDeCine[f, c] = "[ O ]";
        }
    }
}
```

```
// Mostrar la pantalla del cine y la matriz con filas A-E
1 referencia
static void ReflejarMatriz()
{
    Console.WriteLine("================== PANTALLA ==================\n");
    Console.Write("          C1     C2     C3     C4     C5     C6\n\n");

    for (int f = 0; f < 5; f++)
    {
        // Muestra la letra de la fila (A, B, C, D, E)
        Console.Write($"Fila {LetrasFilas[f]}  ");

        for (int c = 0; c < 6; c++)
        {
            Console.Write($" {SalaDeCine[f, c]} ");
        }
        Console.WriteLine("\n");
    }
    Console.WriteLine("Información Asientos: [ O ] = Disponible  |  [ X ] = Ocupado");
}
```

7.2 Módulo de Captura, Validación y Conversión

(ReservarAsiento y ConvertirLetraAIndice): Para evitar desbordamientos de índice (IndexOutOfRangeException), la letra ingresada se convierte a mayúscula (ToUpper()) y se evalúa en ConvertirLetraAIndice(), retornando -1 si no pertenece al rango A-E. Asimismo, se valida con int.TryParse() que la columna sea un entero entre 1 y 6 antes de acceder a la matriz.

```
1 referencia
static void ReservarAsiento()
{
    Console.WriteLine("--- RESERVAR ASIENTO ---\n");

    Console.Write("Ingrese la letra de la fila (A - E): ");
    string entradaFila = Console.ReadLine().ToUpper(); // Convierte a mayúscula (ej: 'a' -> 'A')

    Console.Write("Ingrese el número de columna (1 - 6): ");
    int columna = int.Parse(Console.ReadLine());

    // Convertir letra a índice numérico (A=0, B=1, C=2, D=3, E=4)
    int fila = ConvertirLetraAIndice(entradaFila);

    // Validar límites
    if (fila == -1 || columna < 1 || columna > 6)
    {
        Console.WriteLine("\nError: El asiento seleccionado no existe. (Recuerde: Filas A-E y Columnas 1-6)");
        return;
    }

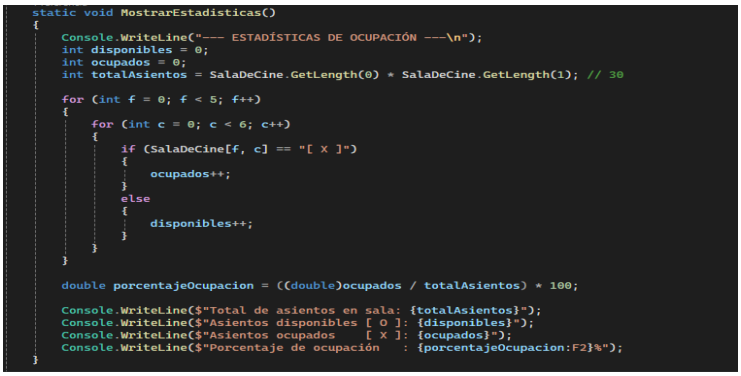
    int c = columna - 1;

    if (SalaDeCine[fila, c] == "[ O ]")
    {
        SalaDeCine[fila, c] = "[ X ]";
        Console.WriteLine($"\n¡Éxito! Asiento {entradaFila}{columna} reservado correctamente.");
    }
    else
    {
        Console.WriteLine($"\nError: El asiento {entradaFila}{columna} ya está ocupado.");
    }
```

```
// Método auxiliar simple para transformar letras A-E a índices 0-4
3 referencias
static int ConvertirLetraAIndice(string letra)
{
    switch (letra)
    {
        case "A": return 0;
        case "B": return 1;
        case "C": return 2;
        case "D": return 3;
        case "E": return 4;
        default: return -1; // Retorna -1 si escriben una letra no válida (ej: 'Z')
    }
}
```

7.3 Módulo de Estadísticas y Porcentaje

(MostrarEstadisticas): Recorre toda la matriz incrementando los contadores ocupados y disponibles según el contenido de cada celda ("[X]" o "[O]"). Posteriormente realiza un casteo explícito a double para calcular el porcentaje exacto: ((double)ocupados / 30) * 100.



```
static void MostrarEstadisticas()
{
    Console.WriteLine("--- ESTADÍSTICAS DE OCUPACIÓN ---\n");
    int disponibles = 0;
    int ocupados = 0;
    int totalAsientos = SalaDeCine.GetLength(0) * SalaDeCine.GetLength(1); // 30

    for (int f = 0; f < 5; f++)
    {
        for (int c = 0; c < 6; c++)
        {
            if (SalaDeCine[f, c] == "[ X ]")
            {
                ocupados++;
            }
            else
            {
                disponibles++;
            }
        }
    }

    double porcentajeOcupacion = ((double)ocupados / totalAsientos) * 100;

    Console.WriteLine($"Total de asientos en sala: {totalAsientos}");
    Console.WriteLine($"Asientos disponibles [ O ]: {disponibles}");
    Console.WriteLine($"Asientos ocupados    [ X ]: {ocupados}");
    Console.WriteLine($"Porcentaje de ocupación   : {porcentajeOcupacion:F2}%");
}
```

8. DIA 5: PRUEBAS REALIZADAS

Caso de Prueba	Entrada	Resultado Esperado	Estado
Reserva válida	Fila `A`, Columna `2`	Cambia `A2` a `[ X ]` y muestra confirmación.	Exitoso
Reserva duplicada	Fila `A`, Columna `2`	Muestra error de asiento ocupado sin sobrescribir.	Exitoso
Entrada inválida	Fila `Z`, Columna `abc`	Bloquea el acceso y muestra mensaje de error controlado.	Exitoso
Liberación de asiento	Fila `A`, Columna `2`	Cambia `A2` de `[ X ]` a `[ O ]`.	Exitoso
Cálculo de estadísticas	3 asientos ocupados	Muestra 3 ocupados, 27 disponibles y `10.00%` de ocupación.	Exitoso

9. CONCLUSIONES INDIVIDUALES

- **Valeria Berenice Alvarado Castillo**: A través de la implementación de CinePI pude aplicar de forma práctica las estructuras de control repetitivas y selectivas para resolver un problema real de gestión de espacios. Además, el trabajo colaborativo con Git y GitHub fue fundamental para integrar los módulos de consulta, reservación y estadísticas de manera ordenada y sin pérdida de código.
- **Rosalia Chávez Lagunas:** El desarrollo de este proyecto me permitió consolidar el manejo de arreglos bidimensionales en C# y comprender cómo estructurar un sistema de consola mediante programación modular. Separar la lógica en funciones independientes facilitó detectar errores de índices y validar entradas del usuario antes de modificar la matriz, logrando un software estable y fácil de mantener.

10. REFERENCIAS

- Microsoft Learn. (2026). *Matrices multidimensionales (Guía de programación de C#)*. Microsoft Documentation. https://learn.microsoft.com/es-es/dotnet/csharp/programming-guide/arrays/multidimensional-arrays
- Joyanes Aguilar, L. (2020). *Fundamentos de programación: Algoritmos, estructura de datos y objetos* (5.ª ed.). McGraw-Hill Interamericana.